

MAREVLO · PRODUCTION RAG

The Vector Database

Where your embeddings live — and the ten ways people quietly break it.

PART I — FOUNDATIONS

Module 4

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

In Module 3 we learned to turn each piece of text into an embedding — a point on a "meaning map." That leaves an obvious question hanging: where do all those points live, and how do we find the nearest ones quickly when a user asks something? If you had a million pieces, you could not compare a question against every one of them by hand, or even in a normal database. You need a special kind of storage built for exactly this job. That storage is the **vector database**, and it is the subject of this module.

This module has two halves. In the first half we build a clear understanding of what a vector database is, the clever trick it uses to search quickly, and how to store and search our own data with it. In the second half — which is the larger half, by your request — we go through the most common ways people break their vector database in practice. The vector store is where the most expensive and most hidden production mistakes live, so learning the faults here pays off more than almost anywhere else in the course.

THE WHOLE MODULE IN ONE SENTENCE

A vector database stores your embeddings and answers one question very, very fast: "which stored pieces are nearest to this one?" Getting it set up correctly is easy to do badly and quietly, which is why most of this module is about the mistakes.

Where we are, and a note on the code

We are still finishing the indexing pipeline. We chunked in Module 2, embedded in Module 3, and now we store those embeddings so they can be searched. As before, all code is written from scratch around our coffee-machine manual, and uses real public libraries whose import paths you should always verify against the version you install. For learning, we use a simple local vector store; the production-grade options come later, in Module 11.

```
# The indexing pipeline -- we now reach the final step:
Load --> Chunk --> Embed -->

# After this, the indexing pipeline is complete and we can build the
# query side (Module 5).
```

SECTION 1

What a vector database is (and why a normal one won't do)

A normal database finds exact matches. A vector database finds nearest neighbours. That difference is everything.

You may already be comfortable with a normal database — the kind that stores rows of data and lets you ask things like "find the customer whose id is 4012" or "find all orders placed in March." Normal databases are superb at *exact* questions like those. But notice what they cannot do: there is no sensible way to ask a normal database "find the rows whose *meaning* is closest to this sentence." Meaning is not an exact match; it is a matter of nearness, and ordinary databases have no notion of nearness in meaning.

A **vector database** is built specifically for that one job. You give it your embeddings — the meaning-map points from Module 3 — and it lets you ask the question that matters for RAG: "given this new point (the user's question), which stored points are the nearest?" It is, at heart, a tool for finding nearest neighbours among a very large number of points, extremely quickly. Everything else it offers is in service of that core ability.

THINK ABOUT IT

Before reading on, ask yourself: if you had one million points and a new query point, what is the most obvious way to find the nearest stored point? Most people answer "compare the query to every stored point and keep the closest." That answer is correct — and it is also exactly the thing a vector database works hard to *avoid* doing. The next section explains why, and what it does instead.

SECTION 2

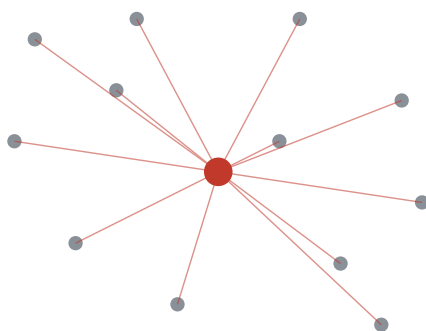
The clever trick: approximate nearest-neighbour search

To be fast at scale, vector databases do not check every point. They take a clever shortcut — and understanding that shortcut prevents a whole family of bugs.

The obvious way to find the nearest point is to compare the query against every single stored point and keep the closest. This is called **exact search**, and it is perfectly accurate. The problem is speed. If you have ten million stored points, every single question requires ten million comparisons. That is far too slow for a real product where users expect an answer in a fraction of a second.

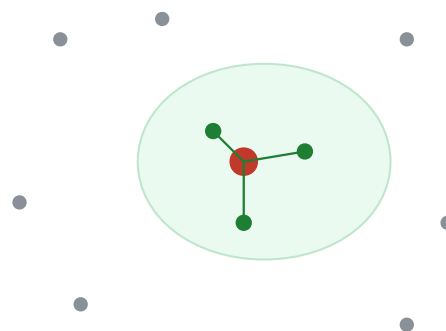
So vector databases use a smarter approach called **approximate nearest-neighbour search**, usually shortened to ANN. Instead of comparing against everything, the database builds a clever index ahead of time — think of it as a network of shortcuts through the meaning map — that lets it jump straight to the promising neighbourhood and only compare against a small handful of nearby points. This is dramatically faster. The word "approximate" is the catch: because it does not check everything, it can occasionally miss the true closest point and return the second-closest instead. In practice the accuracy is very high, and the enormous speed gain is almost always worth the tiny risk.

Exact search: compare to ALL points



accurate, but slow at scale

Approximate search: check a small neighbourhood



nearly as accurate, hugely faster

Figure 1. Exact search compares the query (red) against every stored point. Approximate search jumps to a promising neighbourhood and only compares against a few — far faster, and almost always just as good.

ANALOGY — FINDING THE NEAREST COFFEE SHOP

Suppose you want the nearest coffee shop. The exact method is to measure the distance to every coffee shop in the entire country and pick the smallest — perfectly correct, hopelessly slow. The approximate method is to look only in your own neighbourhood first, where the nearest one almost certainly is. Just occasionally the true nearest is a few steps across a boundary you did not check, and you miss it by a little. For the huge speed saving, that rare small miss is a trade worth making — and that is exactly the bargain a vector database strikes.

Why does this matter for the rest of the module? Because misunderstanding "approximate" causes real confusion. People sometimes notice that a chunk they know is in the database did not come back for a query, and they assume the database is broken or the data is missing. Often it is neither — it is simply the approximate nature of the search, and it can be tuned. Most vector databases give you a dial: search more thoroughly for better accuracy but slower speed, or more shallowly for the reverse. Knowing that dial exists turns a baffling bug into a simple setting.

SECTION 3

Hands-on: storing our chunks

Let us actually create a store, add our manual's chunks to it, and save it to disk so it survives a restart.

We will use a simple local vector store for learning. The important habits we build here — attaching metadata, giving each piece a stable identity, and saving to disk — carry over to every production database later. Read the comments carefully, because several of them point directly at faults we will examine in the second half.

LAB 4.1 — Create a persistent store and add our chunks (with metadata and stable IDs)

```
# Import a local vector store and our embedding model. Verify the exact
# import paths against the versions you install.
from langchain_chroma import Chroma
import hashlib

# Reuse the SAME embedder we built in Module 3. Using a different one
# here than at query time is Fault 4 -- do not.
embedder = make_embedder()

def stable_id(text):
    """A repeatable ID from the content, so re-adding the same
    piece overwrites instead of duplicating (see Fault 1)."""
    return hashlib.sha256(text.encode()).hexdigest()

# Each chunk carries its text AND metadata. The metadata is what lets us
# filter later -- by section, version, or owner. Skipping it is Fault 5.
chunk_texts = [
    "Run a descaling cycle every two months using descaling solution.",
    "Rinse the drip tray and wipe the steam wand after each use.",
    "The machine is covered by a two-year limited warranty.",
]
chunk_metadata = [
    {"section": "descaling", "version": 2025},
    {"section": "cleaning", "version": 2025},
    {"section": "warranty", "version": 2025},
]
chunk_ids = [stable_id(text) for text in chunk_texts]

# Create a store that SAVES TO DISK (persist_directory). An in-memory-only
# store vanishes on restart -- that is Fault 2.
store = Chroma(
    collection_name="coffee_manual",
    embedding_function=embedder,
    persist_directory="./coffee_store",  # survives restarts
)

# Add the chunks. Passing ids makes this safe to re-run.
store.add_texts(texts=chunk_texts, metadatas=chunk_metadata, ids=chunk_ids)
print("Stored", len(chunk_texts), "chunks.")
```

Notice three deliberate choices in that code, each of which prevents a fault we will meet shortly: every chunk has metadata attached, every chunk has a stable identity derived from its content, and the store is written to disk rather than kept only in memory. None of these is optional in a real system, even though every one of them is easy to forget in a quick prototype.

SECTION 4

Searching: scores, thresholds, and filtering

Storing is half the job. Now we search — and we pay attention to the score and the metadata, because both are how we keep answers trustworthy.

Searching the store means handing it a question and asking for the nearest chunks. The most useful version of search also returns a **score** for each result, telling you how close the match was. As we saw in Module 3, that score is not just for ranking; it is a confidence signal. We use it below to decide whether we actually found something, or whether we should admit we did not.

LAB 4.2 — Search with scores, and refuse to answer when nothing is close

```
def search_with_confidence(store, question, k=4, min_score=0.6):
    """Return the nearest chunks, but only if at least one is a real match.

    The min_score check is what stops the system from confidently handing
    back its four least-bad pieces for a question it cannot answer.
    """
    # Ask for the k nearest chunks, each with a similarity score.
    results = store.similarity_search_with_relevance_scores(question, k=k)

    # If even the best match is weak, the answer probably is not here.
    if not results or results[0][1] < min_score:
        return [] # caller should say "I don't know"

    return results

hits = search_with_confidence(store, "How often do I descale?")
for document, score in hits:
    print(f"{score:.2f} {document.page_content}")
```

The second tool we reach for at search time is **metadata filtering**. Recall that every chunk carries metadata such as its section or version. Filtering lets us tell the search "only consider the 2025 version" or "only the warranty section" *before* it looks for nearest neighbours. This is how we stop the system from confidently answering with last year's policy or from a completely unrelated part of the document.

LAB 4.3 — Filter by metadata so we search only the right slice

```
# Tell the search to consider ONLY chunks whose version is 2025.
# The filter is applied DURING the search (pre-filtering), not after,
# which matters a great deal -- see Fault 5b.
results = store.similarity_search(
    "What does the warranty cover?",
    k=4,
    filter={"version": 2025}, # restrict the search to the right slice
)
for document in results:
    print(document.metadata, "->", document.page_content)
```

With storing and searching in hand, the indexing pipeline is complete and we can retrieve relevant chunks for any question. But before we move on to generating answers, we need to spend real time on what goes wrong — because a vector database that *looks* like it is working can be quietly broken in ways that only surface in production.

SECTION 5

The ten faults people actually commit

This is the heart of the module. Each fault below is something teams genuinely do. For each, we show the broken code or setup, explain plainly why it breaks, and give the correction.

Setup faults — F1 rebuilding from scratch (duplicates) · F2 never saving to disk · F3 no stable IDs (can't update or delete) · F4 different embedder than query time · F5 storing without metadata

Search faults — F6 filtering after retrieval instead of before · F7 no relevance threshold · F8 reading the score the wrong way

Scale & safety faults — F9 a learning store in production (no concurrency, no backups) · F10 multi-tenant data with no tenant filter (a data leak)

FAULT 1 — Rebuilding the whole store on every run, doubling everything

THE BROKEN SETUP

```
# Every time the indexing script runs, this creates the store fresh AND
# adds all chunks again -- so after three runs you hold three copies.
store = Chroma.from_texts(chunk_texts, embedder) # no ids, re-adds everything
```

WHY IT BREAKS

Indexing feels like a one-time job, so people re-run the script after any small change without clearing the old data. Now every chunk exists two, three, or four times. At search time the top results are duplicates of one another, so the model receives the same fact repeated instead of several different useful pieces — and you paid to embed everything multiple times. This is the most common "why are all my answers the same sentence?" bug.

THE CORRECTION

```
# Give each chunk a stable ID from its content. Re-adding the same piece
# now overwrites the existing entry instead of creating a duplicate.
ids = [stable_id(text) for text in chunk_texts]
store.add_texts(chunk_texts, metadatas=chunk_metadata, ids=ids) # safe to re-run
```


FAULT 2 — Never saving the store to disk

THE BROKEN SETUP

```
# No persist_directory, so the store lives only in memory.
store = Chroma(collection_name="coffee_manual", embedding_function=embedder)
# Works in this session... then the program restarts and it is all gone.
```

WHY IT BREAKS

A store kept only in memory disappears the moment your program stops. In a notebook this is invisible because the session stays alive, so everything seems fine. Then you deploy, the server restarts overnight, and in the morning every question returns nothing because the index is empty — and you have to re-embed your entire collection to recover, which costs time and money.

THE CORRECTION

```
# Give it a folder to save into, so the index survives restarts.
store = Chroma(
    collection_name="coffee_manual",
    embedding_function=embedder,
    persist_directory="./coffee_store",  # now it is durable
)
```

FAULT 3 — No stable IDs, so you can never update or delete a document

THE BROKEN SETUP

```
# Chunks added with random or no IDs. When a document changes, you have
# no way to find and replace its old chunks -- they linger forever.
store.add_texts(chunk_texts)  # IDs auto-generated and forgotten
```

WHY IT BREAKS

Documents change. A policy is updated, a page is corrected, a product is discontinued. If you never gave your chunks stable identities, you cannot point at "the old chunks for this document" to remove or replace them. The outdated chunks stay in the store and keep getting retrieved, so the system confidently serves stale information — and you cannot clean it up without rebuilding from scratch.

THE CORRECTION

```
# Stable IDs let you update and delete precisely.
ids = [stable_id(text) for text in chunk_texts]
store.add_texts(chunk_texts, ids=ids)

# Later, when a document is removed, delete exactly its chunks:
store.delete(ids=[stable_id(old_chunk)])
```

FAULT 4 — Using a different embedding model than you used to build the store

THE BROKEN SETUP

```
# Built the store with one model...
store = Chroma(embedding_function=OpenAIEmbeddings(model="text-embedding-3-small"),
                persist_directory="./coffee_store")
# ...then re-opened it later with a DIFFERENT model.
store = Chroma(embedding_function=OpenAIEmbeddings(model="text-embedding-3-large"),
                persist_directory="./coffee_store") # mismatch!
```

WHY IT BREAKS

This is the Module 3 rule applied to the database. The store holds embeddings made by the first model, on the first model's "meaning map." Querying it with a second model means comparing points from two unrelated maps. Nothing errors; retrieval just returns near-random chunks, and because the store still "works," the cause is maddeningly hard to spot. It often sneaks in when someone changes a config or upgrades a library that quietly changes the default model.

THE CORRECTION

```
# Pin one embedder and reuse it for building AND opening the store.
# If you ever truly must switch models, you have to rebuild the whole
# store from your source documents -- there is no shortcut.
embedder = make_embedder() # the single source of truth
store = Chroma(embedding_function=embedder, persist_directory="./coffee_store")
```

FAULT 5 — Storing chunks with no metadata, so you can never filter

THE BROKEN SETUP

```
# Just the text, nothing about where it came from or which version it is.
store.add_texts(chunk_texts) # no metadatas
```

WHY IT BREAKS

Without metadata, every chunk is an anonymous piece of text. You cannot later say "only the current version," "only this product," or "only this customer's documents," because the store has no facts to filter on. The single most common consequence is the system confidently answering from an outdated or wrong-context chunk, simply because it had no way to exclude it. Adding metadata after the fact means re-indexing, so it is far cheaper to attach it from the start.

THE CORRECTION

```
# Attach the facts you will want to filter on, at storage time.
metadatas = [{"section": "descaling", "version": 2025, "product": "coffee_x1"},
              {"section": "cleaning", "version": 2025, "product": "coffee_x1"}]
store.add_texts(chunk_texts, metadatas=metadatas, ids=ids)
```

FAULT 6 — Filtering after retrieval instead of before it

THE BROKEN SETUP

```
# Retrieve the 4 nearest from EVERYTHING, then throw away the ones that
# do not match the filter. You may be left with 1 result, or none.
results = store.similarity_search(question, k=4)
results = [r for r in results if r.metadata["version"] == 2025] # too late
```

WHY IT BREAKS

This is subtle and very common. When you retrieve first and filter afterward, the search picked the four nearest chunks from the *whole* store without caring about your filter. If three of those four happened to be the 2024 version, you discard them and are left with a single result — or zero — even though plenty of relevant 2025 chunks existed. You asked for four good answers and the filtering quietly starved you down to one. The fix is to filter *during* the search so all four come from the right slice.

THE CORRECTION

```
# Pass the filter INTO the search so it only ever considers matching
# chunks. Now all k results come from the slice you wanted.
results = store.similarity_search(question, k=4, filter={"version": 2025})
```

FAULT 7 — No relevance threshold, so it answers questions it cannot answer

THE BROKEN SETUP

```
# Always take the top chunks and answer, even when the best match is weak.
results = store.similarity_search(question, k=4)
answer = generate(question, results) # answers even total nonsense
```

WHY IT BREAKS

A nearest-neighbour search always returns *something* — the nearest points exist even when none of them is actually relevant. If you never check how close the best match really is, then for a question your documents do not cover, the system hands the model its four least-bad-but-still-irrelevant chunks, and the model produces a confident, fabricated answer. This is the most reputationally damaging failure a RAG system can have, and it is entirely preventable.

THE CORRECTION

```
# Check the best score. If even the closest match is weak, refuse.
results = store.similarity_search_with_relevance_scores(question, k=4)
if not results or results[0][1] < 0.6: # tune this number on your data
    answer = "I don't have information on that."
else:
    answer = generate(question, [r[0] for r in results])
```

FAULT 8 — Reading the score the wrong way round

THE BROKEN SETUP

```
# Some stores return a DISTANCE (lower = closer), not a similarity.  
# Thresholding as if higher is always better filters out your BEST hits.  
good = [r for r in results if r.score > 0.6] # may keep the worst, drop the best
```

WHY IT BREAKS

Different stores report closeness differently. Some give a similarity, where a higher number means closer. Others give a distance, where a lower number means closer. If you assume the wrong direction, your threshold does the exact opposite of what you intended — keeping the least relevant chunks and discarding the most relevant — and the system feels broken for no visible reason. A two-minute check of your store's convention prevents hours of confusion.

THE CORRECTION

```
# Confirm which your store returns, then compare in the right direction.  
# Prefer the explicit "relevance score" helper, where higher = better,  
# so the direction is unambiguous.  
results = store.similarity_search_with_relevance_scores(question, k=4)  
good = [doc for doc, score in results if score > 0.6] # higher = better, clearly
```

FAULT 9 — Running a learning-grade store in production

THE BROKEN SETUP

```
# A simple local store, fine for one person in a notebook, now serving  
# real traffic from many users at once.  
store = Chroma(persist_directory="./coffee_store") # under heavy concurrent load
```

WHY IT BREAKS

A store meant for learning is not built for many users hitting it at the same time, for guaranteed durability, or for backups. Under real concurrent traffic it can slow down, lock up, or corrupt, and when it does there is no backup to restore from. The system that ran flawlessly for one person in a notebook falls over the first busy day in production, and the data may not be recoverable.

THE CORRECTION

```
# For production, move to a store built for concurrency, durability, and  
# backups -- for example Postgres with the pgvector extension, which we  
# set up properly in Module 11. The interface is similar; the operational  
# guarantees are not.  
# (Build the eval set first so you can confirm quality matches after moving.)
```

FAULT 10 — Putting many customers' data in one collection with no tenant filter

THE BROKEN SETUP

```
# Every customer's documents share one collection, and search has no  
# filter for who is asking. Relevance alone decides what comes back.  
results = store.similarity_search(question, k=4) # can return ANOTHER customer's  
chunk
```

WHY IT BREAKS

This is the most dangerous fault in the whole module, because it is not a quality bug — it is a data breach. A nearest-neighbour search returns whatever is most *relevant*, with no notion of who is allowed to see it. If customer A asks a question and customer B's document happens to be the closest match, the system will hand customer A customer B's confidential content. Relevance is not permission. In any system serving more than one customer, you must restrict the search to the asking customer's own data, every single time.

THE CORRECTION

```
# Tag every chunk with its owner at storage time, then ALWAYS filter by  
# the current user at search time. Build this in first, not last.  
store.add_texts(texts, metadatas=[{"owner": "customer_A"} for _ in texts], ids=ids)  
  
# At query time, never search without the owner filter:  
results = store.similarity_search(question, k=4, filter={"owner": current_user_id})
```

SECTION 6

A checklist for a healthy store

If you can answer "yes" to each of these before you ship, you have avoided the ten faults above.

Use the questions below as a final pass over your vector database setup. Each one maps directly to a fault we examined, and going through them takes only a few minutes — far less time than chasing any one of these problems after launch.

Is my indexing safe to run twice without creating duplicates?

Yes if every chunk has a stable ID derived from its content (defeats Fault 1 and Fault 3).

Does my store survive a restart?

Yes if it writes to disk or to a real database, not memory alone (defeats Fault 2).

Am I using exactly one embedding model for both building and querying?

Yes if a single shared embedder is the only one in the codebase (defeats Fault 4).

Can I filter my searches by version, product, and owner?

Yes if every chunk carries that metadata, attached at storage time (defeats Fault 5).

Are my filters applied during the search, not after it?

Yes if the filter is passed into the search call itself (defeats Fault 6).

Will my system admit when it does not know?

Yes if a relevance threshold can return "I don't know," and I read the score in the correct direction (defeats Fault 7 and Fault 8).

Is my production store built for concurrency, durability, and backups?

Yes if I moved off the learning store before launch (defeats Fault 9).

Can one customer ever see another customer's data?

Safe only if every search is filtered by the asking user's identity, with no exceptions (defeats Fault 10 — the breach).

Checkpoint: try it yourself

This exercise makes the faults real by letting you trigger and then fix one of the most common ones with your own hands. It takes about twenty-five minutes.

1. Build a small store from five chunks of your own, giving three of them the metadata `{"version": 2024}` and two of them `{"version": 2025}`.
2. First, commit Fault 6 on purpose: search for something with `k=4`, then filter the results down to version 2025 afterwards. Count how many results survive. You will often be left with fewer than you asked for.
3. Now do it the right way: pass `filter={"version": 2025}` into the search itself, again with `k=4`. Count the results. You should reliably get the full set from the correct slice.

4. Finally, search for something your five chunks clearly do not cover, and look at the top score. Notice how low it is — that low score is exactly the signal a relevance threshold (Fault 7) uses to decide the honest answer is "I don't know."

WHAT YOU SHOULD NOTICE

Doing Fault 6 by hand makes the pre-filter-versus-post-filter difference unforgettable in a way that reading about it never could. You felt the search starve down to one result, and you felt it fixed. That is the kind of mistake that, once you have made it deliberately in a safe exercise, you will never make accidentally in production.

What to carry into Module 5

THE SIX THINGS TO REMEMBER

One, a vector database exists to answer one question fast: "which stored pieces are nearest to this one?" **Two**, it stays fast at scale by using approximate search, which trades a tiny, tunable bit of accuracy for an enormous speed gain. **Three**, always attach metadata and stable IDs, and always save to disk — these three habits prevent half the faults in this module. **Four**, filter *during* the search, not after, and use the score as a confidence signal so the system can honestly say "I don't know." **Five**, a learning store is not a production store — move to one built for concurrency, durability, and backups before you launch. **Six**, relevance is never permission: in any multi-customer system, every single search must be filtered by who is asking, or you have a data breach.

The indexing pipeline is now complete: we can load, chunk, embed, store, and search. In Module 5 we finally build the query side — taking the chunks our store returns, handing them to a language model with a careful prompt, and producing a grounded answer. Everything we have built so far has been preparing the ground for that one moment, and the quality of the answer will depend on every step we took to get here.